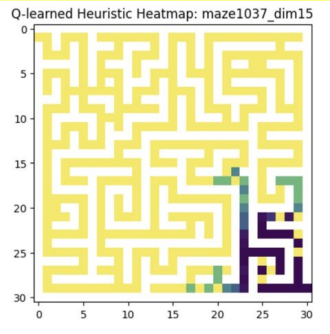
June
2025

Optimizing Heuristic Functions for Grid Mazes via Q-Learning





CS221 Final Project

Brooke Ballhaus, Riya Narain

Confidential

Motivation

Real-world systems (autonomous vehicles, robotics) need fast, optimal pathfinding

A* search is optimal but slow with weak heuristics

Manhattan/Euclidean are admissible but naive — ignore walls or edge weights

Can we learn a smarter heuristic that makes A* faster without losing optimality?

Confidential

Problem Definition

Learn a cost-to-go heuristic h(s) for use in A* search on a grid

Must be:

- Admissible (never overestimates cost)
- Reusable across many start–goal queries
- Faster than hand-coded baselines

Environment: Imperfect mazes with loops, multiple paths, and weighted edges

Confidential

Challenges

Q-learning isn’t designed to generate heuristics directly

Need to reverse train from goal and shape rewards carefully

Learned values must propagate far enough to be useful

Must tune several hyperparameters (step size, ε, episode length, etc.)

Need to balance training cost vs. query-time savings

Approach

Train Q-learning backward from the goal

Reward structure: -1 per step, 0 at sampled start states

Use Bellman update to learn Q-values

Convert to heuristic: $h(s) = -\max_a Q(s, a)$

Tune hyperparameters with regression-based search

Confidential

Dataset

1,500 imperfect mazes from Kaggle

Sizes: 10×10 to 150×150

Imperfect structure = loops, multiple paths → heuristic quality matters

Split: 20% training, 80% evaluation

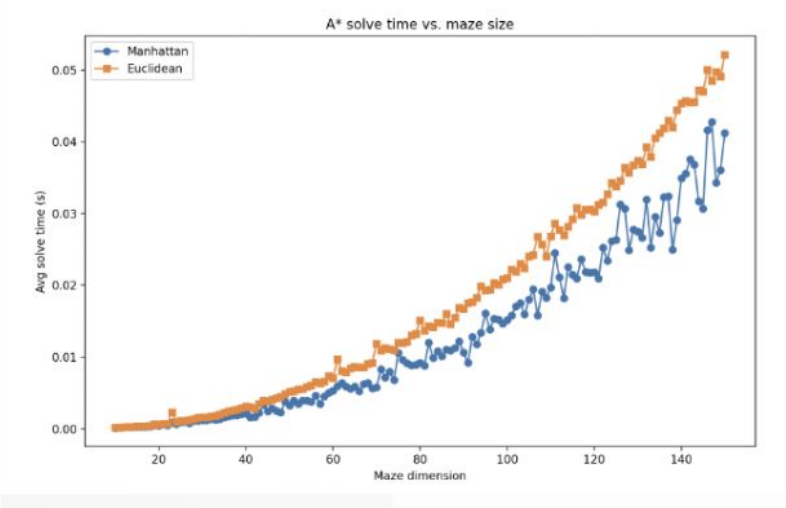
Baseline

Manhattan Heuristic

$$h_{\text{Man}}(s) = |x_s - x_g| + |y_s - y_g|$$

Euclidean Heuristic:

$$h_{\text{Euc}}(s) = \sqrt{(x_s - x_g)^2 + (y_s - y_g)^2}$$



Confidential

Evaluation Metrics

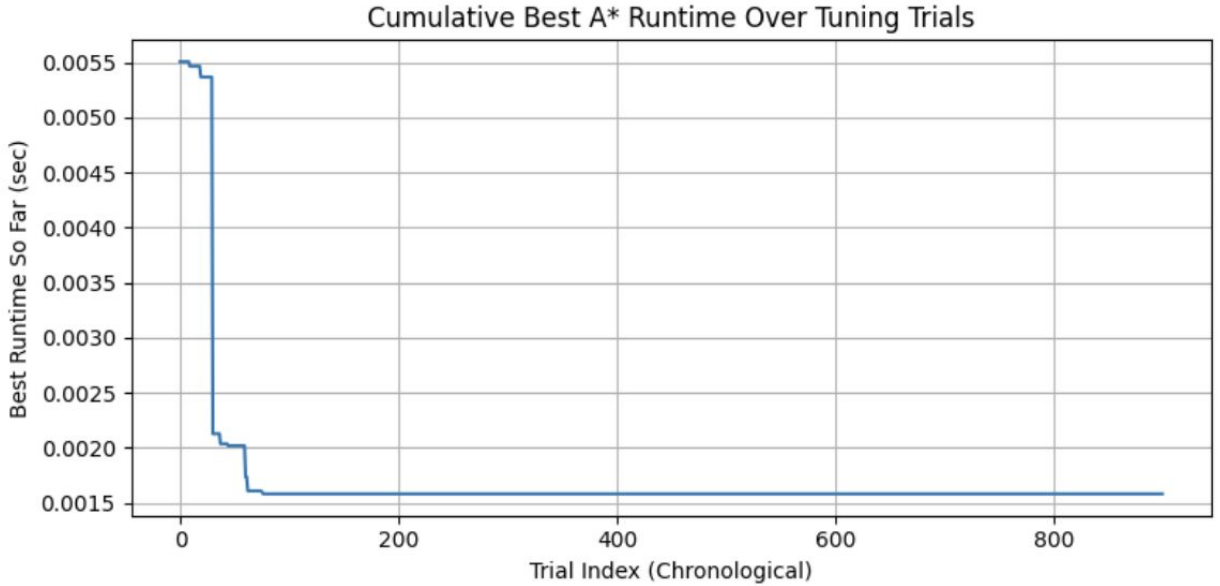
Wall-clock runtime (summed over 3 starts)

Path length (to ensure optimality)

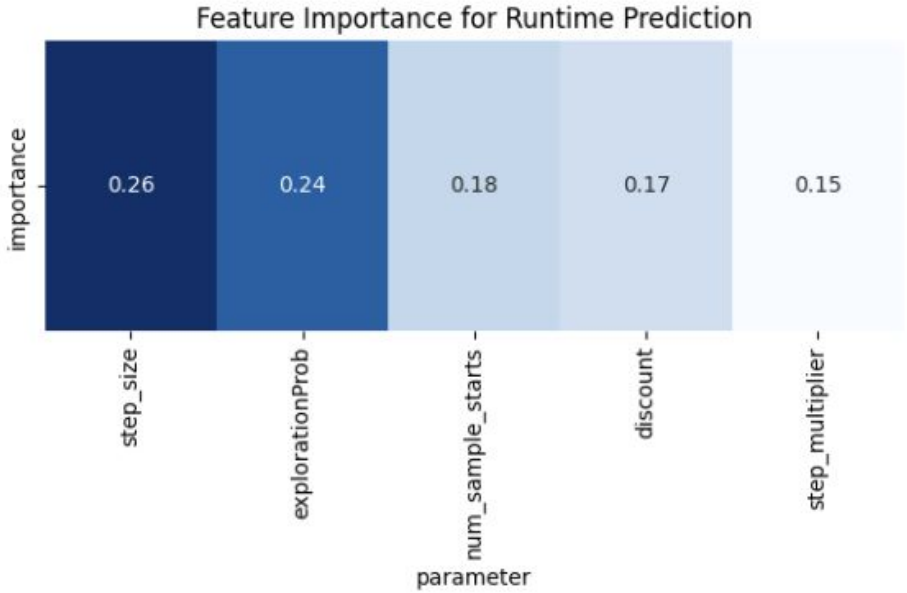
Nodes expanded

Relative runtime improvement vs. better baseline

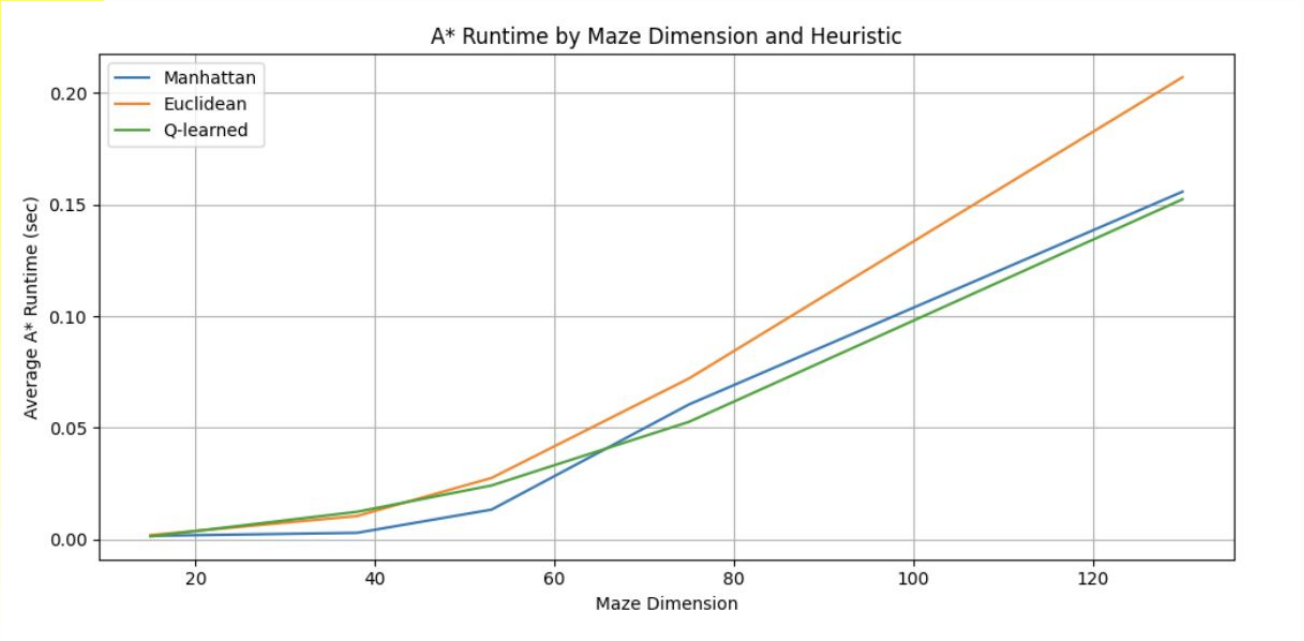
Results


Cumulative Best A* Runtime Over Tuning Trials
Best Runtime So Far (sec)
0.0055
0.0050
0.0045
0.0040
0.0035
0.0030
0.0025
0.0020
0.0015
0
200
400
600
800
Trial Index (Chronological)

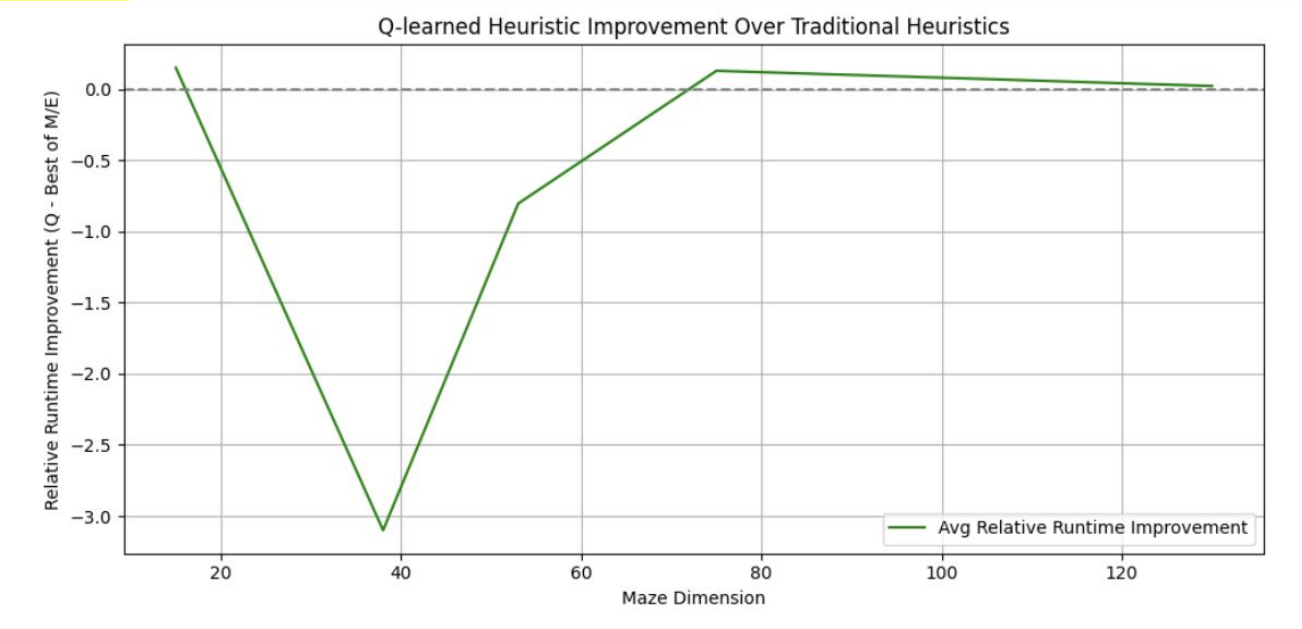
Results


Feature Importance for Runtime Prediction
importance
0.26
0.24
0.18
0.17
0.15
step_size
explorationProb
num_sample_starts
discount
step_multiplier
parameter

Results


A* Runtime by Maze Dimension and Heuristic
Manhattan
Euclidean
Q-learned
Average A* Runtime (sec)
0.00
0.05
0.10
0.15
0.20
20
40
60
80
100
120
Maze Dimension

Results


Q-learned Heuristic Improvement Over Traditional Heuristics
Relative Runtime Improvement (Q - Best of M/E)
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
20
40
60
80
100
120
Maze Dimension
Avg Relative Runtime Improvement

Confidential

Analysis

Q-learned outperforms in complex mazes

Struggles in small mazes (undertrained or flat values)

Q-learned adapts to wall layout

Unlike Manhattan, learns shortcuts and avoids dead ends